

Multimedia Computing

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University in Tashkent.

Email: minhaz.ahmed@gmail.com

Contents

- ▶ What is segmentation
- ▶ edge-based segmentation
 - ▶ Point detection
 - ▶ Line detection
 - ▶ Edge detection
- ▶ Human vision : Grouping and Gestalt
- ▶ Important Application:
 - ▶ Background subtraction
 - ▶ Shot Boundary Detection
 - ▶ Interactive Segmentation
 - ▶ Clustering based segmentation

Image segmentation

- ▶ Image segmentation is the identification and isolation of an image into regions that correspond to structural units

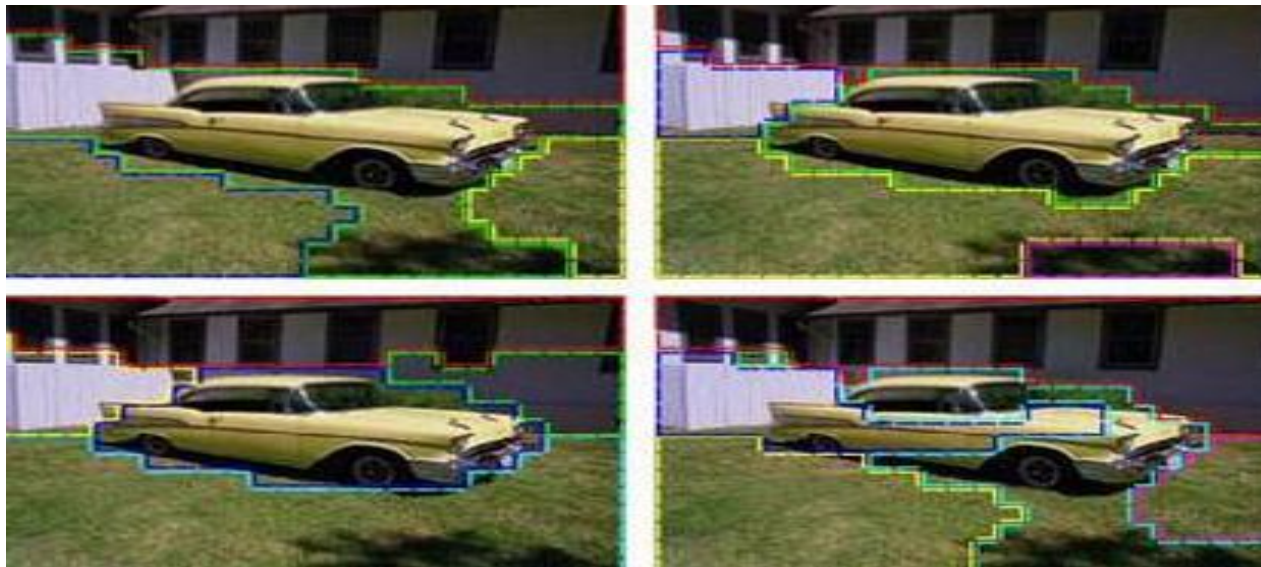


Image segmentation

Image segmentation is the process of partitioning a digital image into multiple regions and extracting the meaningful region which is known as Region of Interest(ROI)

- ☐ Region of Interest (ROI) vary with applications
- ☐ In fact no single universal segmentation algorithm exists for segmenting the ROI in all images
- ☐ Therefor many segmentation algorithms need to apply and pick that algorithm which performs the best for given requirement

What's Segmentation?

- ▶ To recognize objects in an image.
- ▶ Too many pixels to handle each individually
- ▶ There should be relatively few components in the representation computed for typical pictures.
- ▶ These components should be suggestive.
- ▶ Obtain a compact representation from an image/motion sequence/set of tokens
- ▶ Should support application
- ▶ Broad theory is absent at present

Image segmentation

▶ Problem

- ▶ Summarizing video
- ▶ Finding people
- ▶ Finding buildings in satellite images
- ▶ Searching a collection of images

edge-based segmentation

- ▶ There are three basic types of gray-level discontinuities in a digital image: points, lines, and edges
- ▶ The most common way to look for discontinuities is to run a mask through the image.
- ▶ We say that a point, line, and edge has been detected at the location on which the mask is centered if $|R| \geq T$, where $R = w_1z_1 + w_2z_2 + \dots + w_9z_9$

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

Classification of the Segmentation techniques

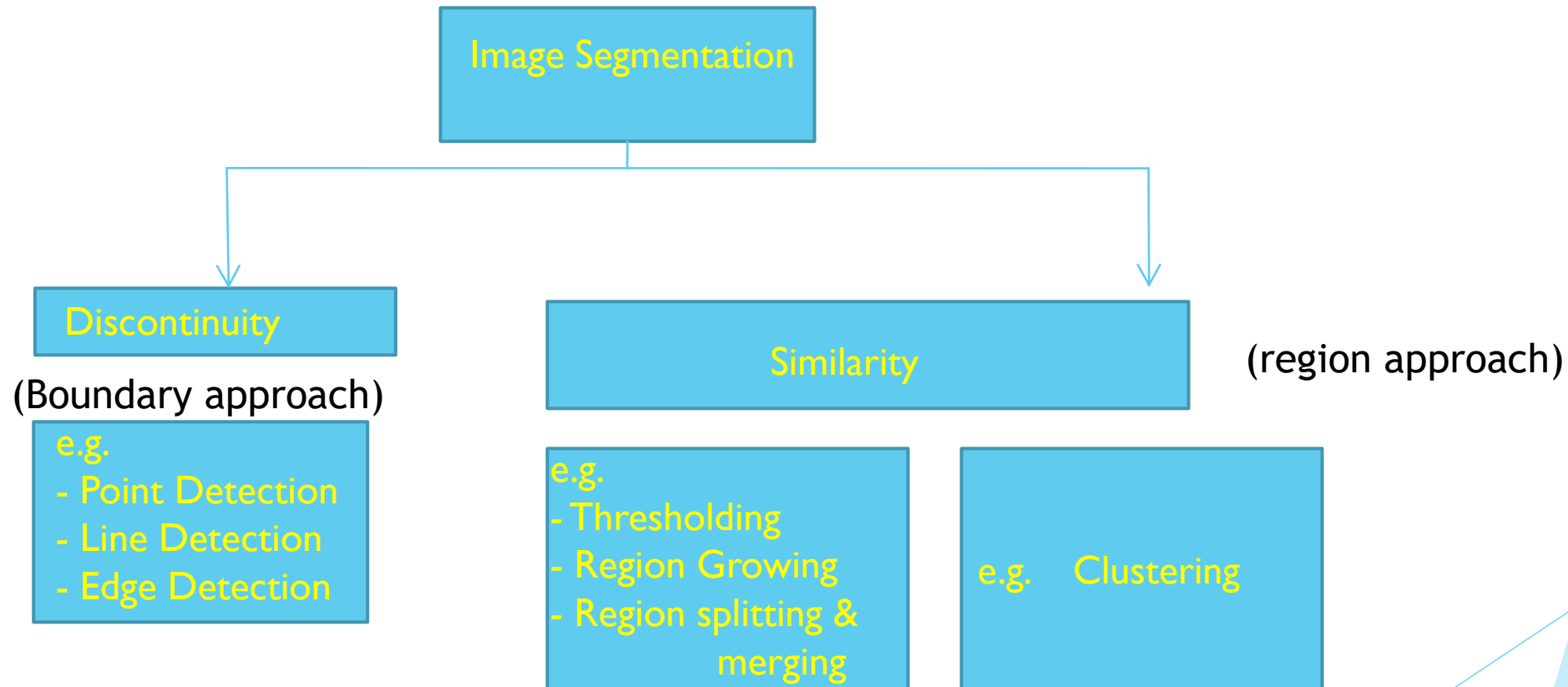
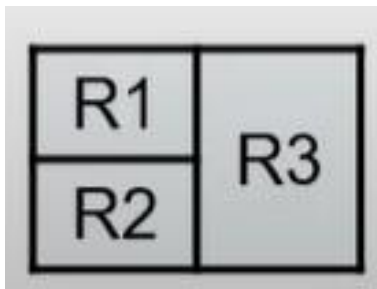


Image segmentation

An image can be portioned into many regions $R_1, R_2, R_3 \dots R_n$

R_1	R_{21}	R_{22}
	R_{23}	R_{24}
R_3	R_4	



10	10	20	20	20
10	10	20	20	20
10	10	20	20	20
30	30	20	20	20
30	30	20	20	20

10	10	20	20	20
10	10	20	20	20
10	10	20	20	20
30	30	20	20	20
30	30	20	20	20

Image segmentation

- ▶ Let R represent the entire image region and segmentation is partitioning R into n subgroups R_i

- $\bigcup_{i=1}^n R_i = R$

- R_i should be connected region $i=1,2,3,\dots,n$

- $R_i \cap R_j = \emptyset$ (for all i and j): $i \neq j$

- $P(R_i) = \text{TRUE}$ for $i = 1,2,3, \dots, n$

- $P(R_i \cup R_j) = \text{FALSE}$ for $i \neq j$

Here $P(R_i)$ is a predicate that indicates some property over the region.

Classification of the Segmentation techniques (General ideas)

► Tokens

- whatever we need to group (pixels, points, surface elements, etc., etc.)

► Top down segmentation

- tokens belong together because they lie on the same object

► Bottom up segmentation

- tokens belong together because they are locally coherent

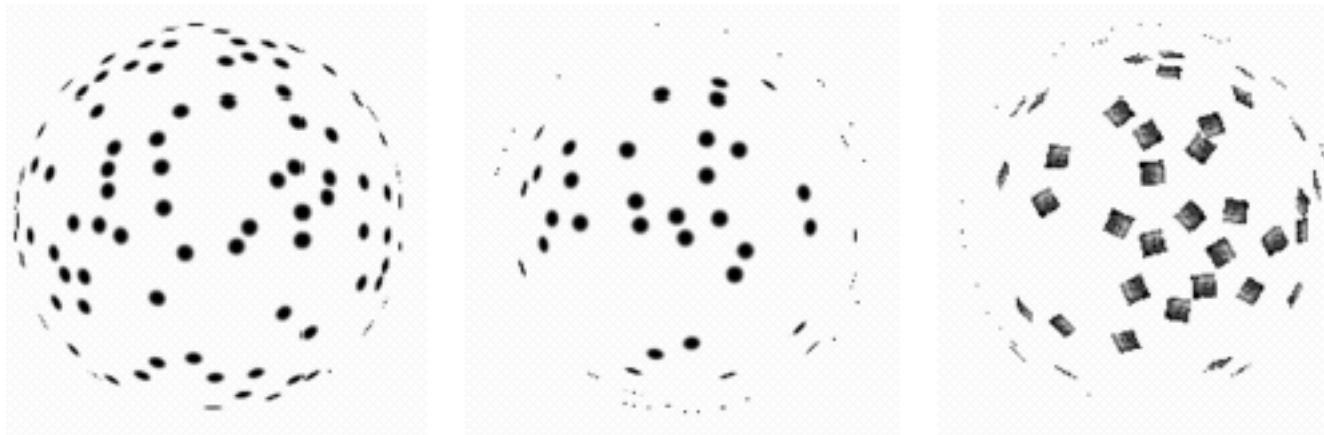


Figure 14.1

Point Detection

- ▶ Based on Masking...

-1	-1	-1
-1	8	-1
-1	-1	-1

- ▶ Find response R.
- ▶ The emphasis is strictly to detect points. That is, differences those are large enough to be considered as isolated points.
- ▶ So, compare and separate based on $|R| > T$
- ▶ Where R = Response of convolution
T = Non negative threshold value

Point Detection(Example)

Original

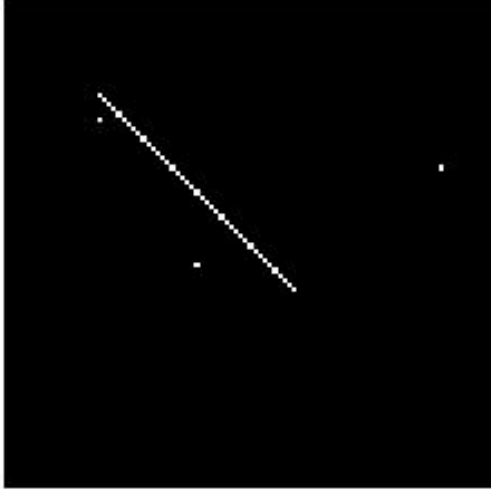
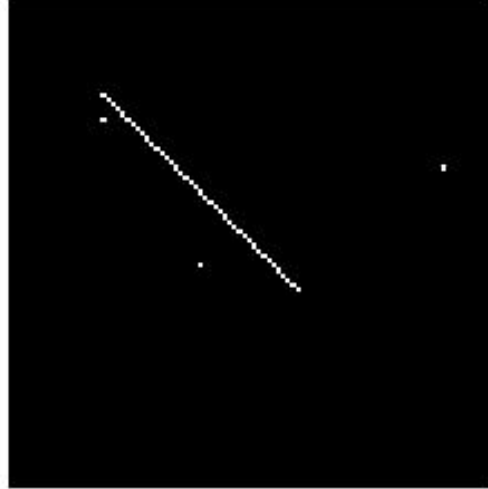


Image after applying mask



Thresholded image by $T=8$



Practice code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an image (grayscale)
image = cv2.imread('test_point.jpg', cv2.IMREAD_GRAYSCALE)

# Define the point detection kernel (Laplacian filter)
# This kernel will detect points with high intensity contrast.
#kernel = np.array([[0, 1, 0],
#                   [1, -4, 1],
#                   [0, 1, 0]])
kernel = np.array([[ -1, -1, -1],
                   [ -1,  8, -1],
                   [ -1, -1, -1]])

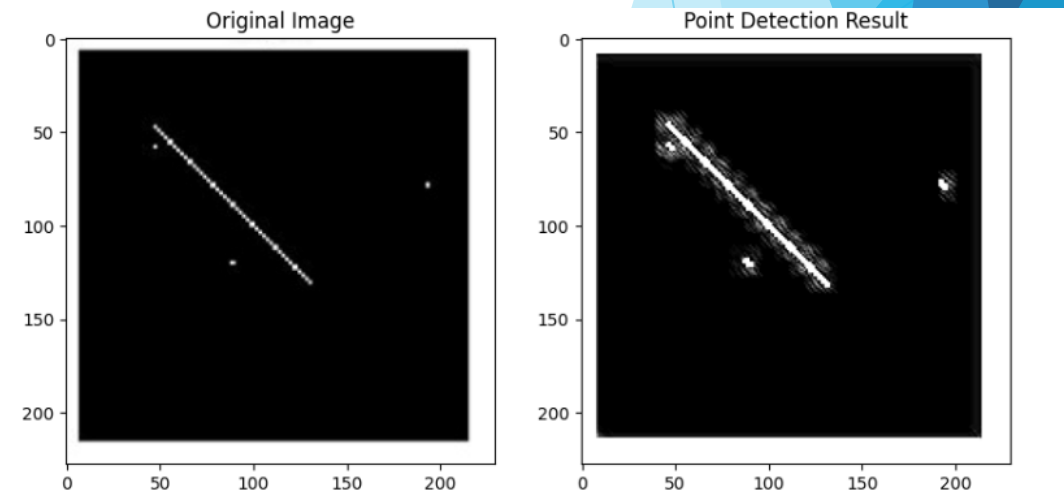
# Apply the filter using convolution
point_detected_image = cv2.filter2D(image, -1, kernel)

# Display the original and point-detected images
plt.figure(figsize=(10,5))

plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')

plt.subplot(1, 2, 2)
plt.title('Point Detection Result')
plt.imshow(point_detected_image, cmap='gray')

plt.show()
```



Line Detection

Horizontal Line

-1	-1	-1
2	2	2
-1	-1	-1

45 degree inclined Line

-1	-1	2
-1	2	-1
2	-1	-1

Vertical Line

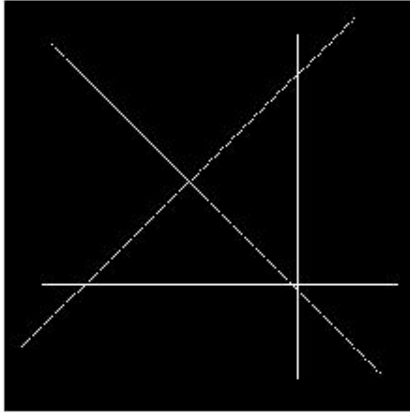
-1	2	-1
-1	2	-1
-1	2	-1

-45 degree inclined Line

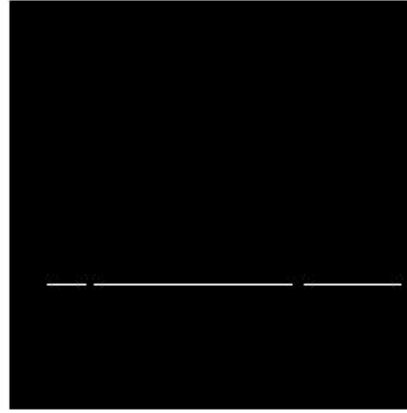
2	-1	-1
-1	2	-1
-1	-1	2

Line Detection

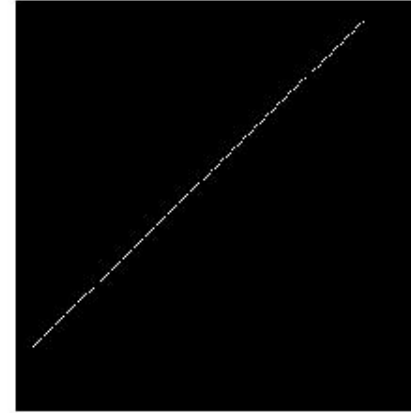
Original Image



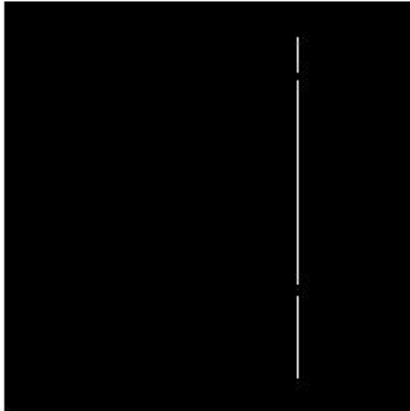
Horizontal line detection



45 degree inclined line detection



Vertical line detection

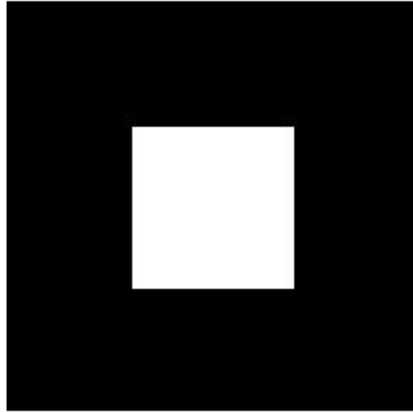


135 degree inclined line detection

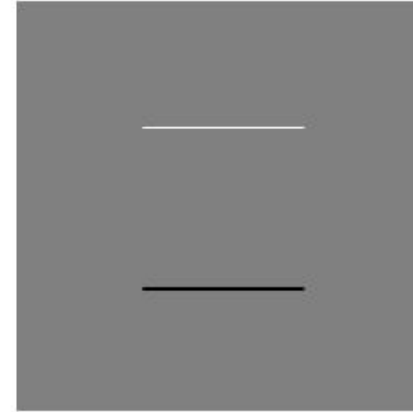


Edge Detection(Example)

Original Image



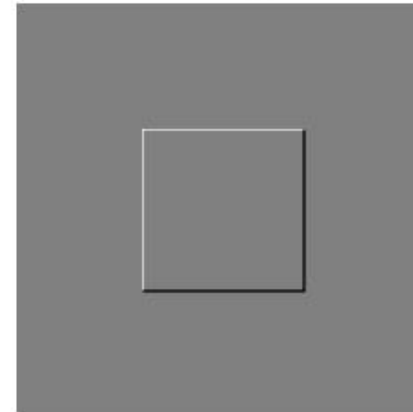
Sobel-Horizontal Edge



Sobel-Vertical Edge



Sobel-Edge



Combined Detection

- ▶ Multi-mask formulation makes possible development of a method to determine whether a pixel is most likely to be an **isolated point** or part of a **line** or an **edge**.

Edge Linking and Boundary Detection

- ▶ Intensity discontinuity can be utilized to find boundary.
- ▶ The lagging part of boundary detection using intensity discontinuity is that the boundary may not be completely defined because of
 - ▶ Noise
 - ▶ Breaks in boundary due to non-uniform illumination
- ▶ So, after edge detection, edge linking process is carried out to assemble edge pixels into meaningful boundary

Need of Edge Linking



The boundary is not complete in edge detection (bottom figure). Required edge linking.



1) Edge Linking - Local Processing

- ▶ Analyze every pixel in small neighborhood that has undergone edge detection.
- ▶ For same characteristics (point is on same edge or not), two principal properties used are
 - ▶ Strength of response of the gradient operator

$$|\nabla f(x, y) - \nabla f(x', y')| \leq T$$

- ▶ The **direction** of gradient.

$$|\alpha(x, y) - \alpha(x', y')| \leq A$$

Segmentation by Clustering (chp-9)

- ▶ **Data reduction** - obtain a compact representation for *interesting* image data in terms of a set of components
- ▶ Find components that belong together (form **clusters**)
- ▶ Frame differencing - Background Subtraction and Shot Detection

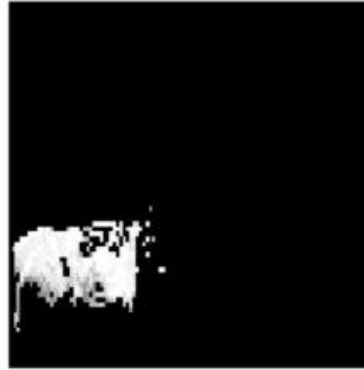
Segmentation by Clustering



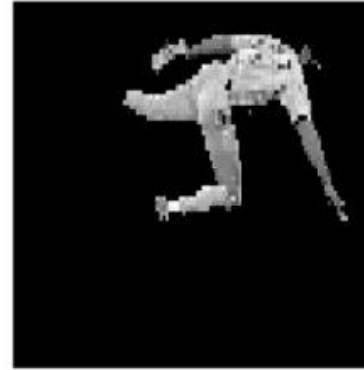
Segmentation by Clustering



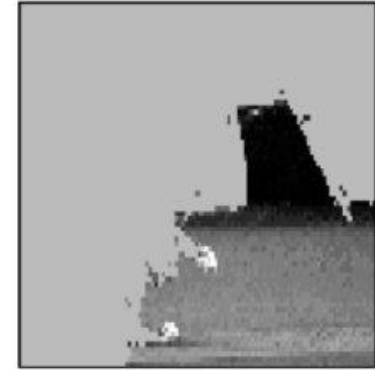
(a)



(b)



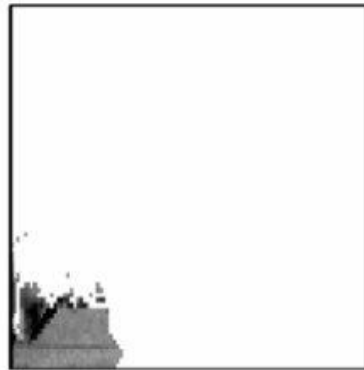
(c)



(d)



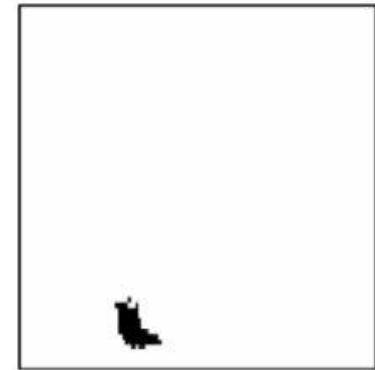
(e)



(f)

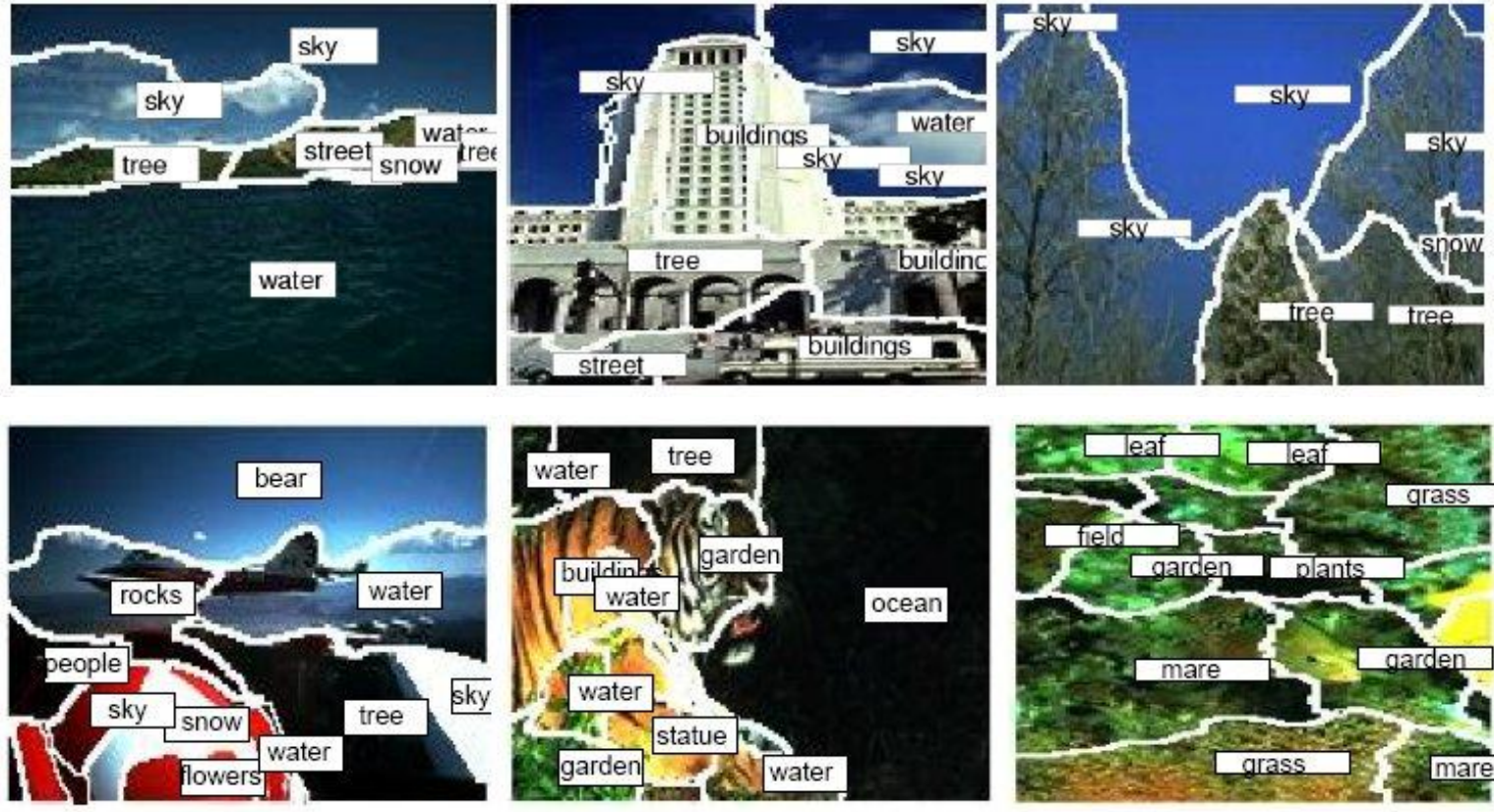


(g)



(h)

Segmentation by Clustering



From: Object Recognition as Machine Translation, Duygulu, Barnard, de Freitas, Forsyth, ECCV02

Examples of segmentation problem

- ▶ Forming image segment
 - ▶ To decompose an image into super-pixels, image regions that have roughly coherent color and texture.
- ▶ Fitting lines to edge points
 - ▶ There are a number of reasons it might be useful to fit a set of lines to a set of lines to a set of points.
- ▶ Fitting a fundamental matrix to a set of feature points
 - ▶ Assume we have two views of a set of feature points

Segmentation as Clustering

▶ Segmentation

- ▶ To determine which components of a data set naturally belong together.
- ▶ This is a problem known as clustering

▶ Cluster's two ways

- ▶ Partitioning
- ▶ Grouping

Segmentation as Clustering

▶ Partitioning

- ▶ Decompose an image into regions that coherent color and texture inside them
- ▶ Decompose an image into extended blobs, consisting of regions that have coherent color, texture, and motion and look like limb segment
- ▶ Take a video sequence and decompose it into shots

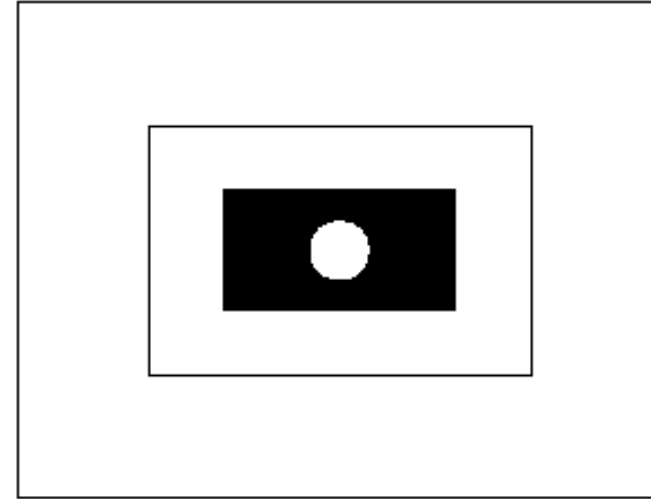
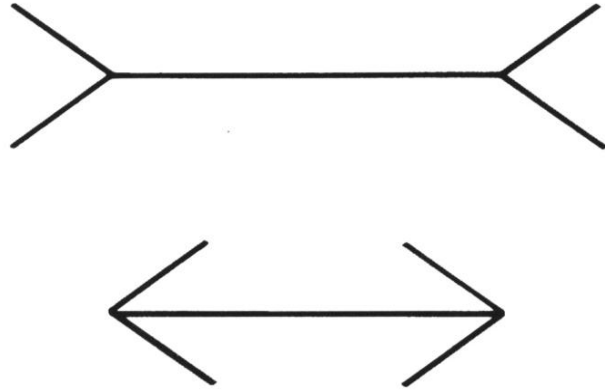
▶ Grouping

- ▶ Collecting together tokens that, taken together, form a line
- ▶ Collecting together tokens that seem to share a fundamental matrix

Human Vision: Grouping and Gestalt

- ▶ A key feature of the human vision system is that context affects how things are perceived
- ▶ Basic ideas of grouping in humans
- ▶ Gestalt properties
 - ▶ elements in a collection of elements can have properties that result from relationships (Müller-Lyer effect)
 - ▶ Gestalt qualities
 - ▶ A series of factors affect whether elements should be grouped together
 - ▶ Gestalt factors

Grouping and Gestalt



The famous Müller-Lyer illusion; the point is that the horizontal bar has properties that come only from its membership in a group (it looks shorter in the lower picture, but is actually the same size)

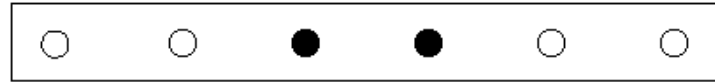
Grouping and Gestalt



Not grouped



Proximity



Similarity



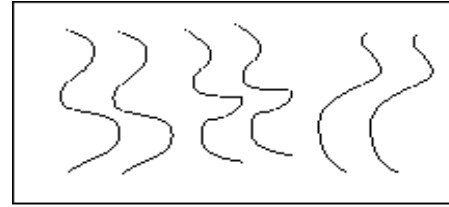
Similarity



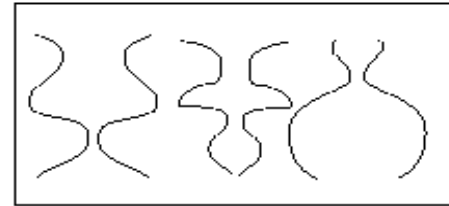
Common Fate



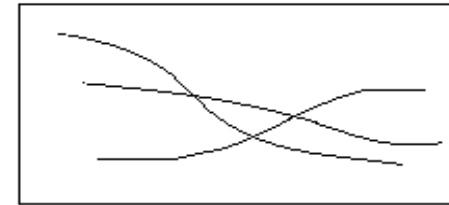
Common Region



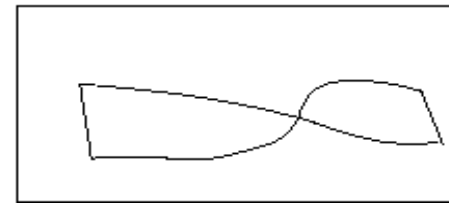
Parallelism



Symmetry

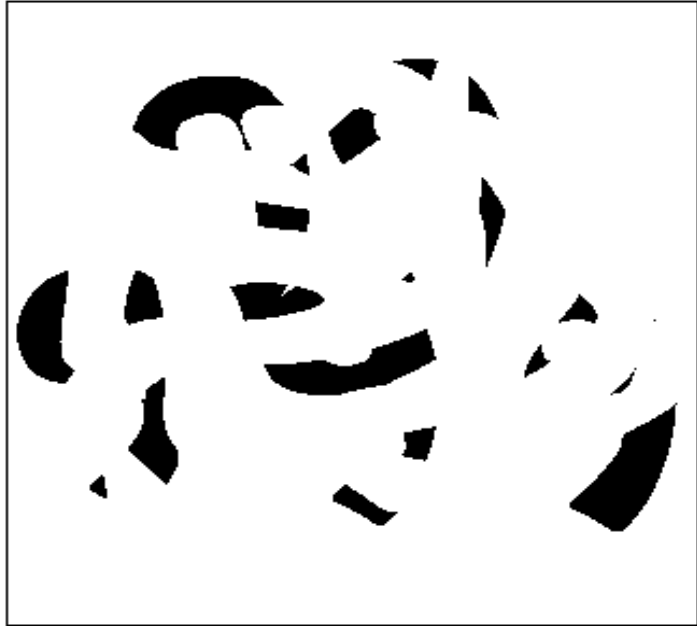


Continuity



Closure

Grouping and Gestalt



A collection of digits



Quite clearly some occluded

Occlusion appears to be an important cue in grouping
Most people find it hard to read the 5 numerals in

Grouping and Gestalt



One is the illusory contour . Here a set of tokens suggests the presence of an object most of whose contour has no contrast

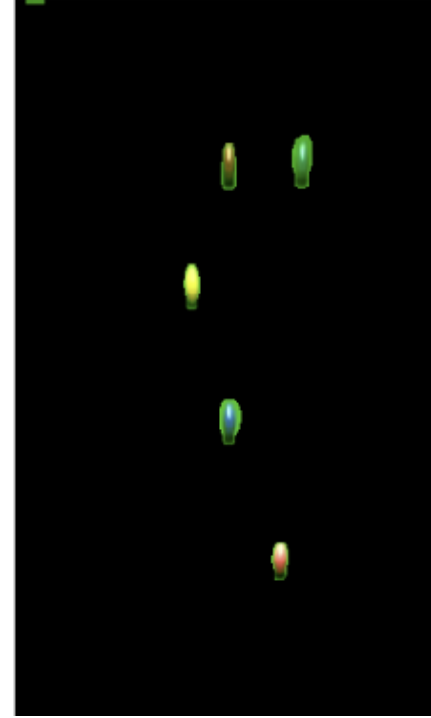
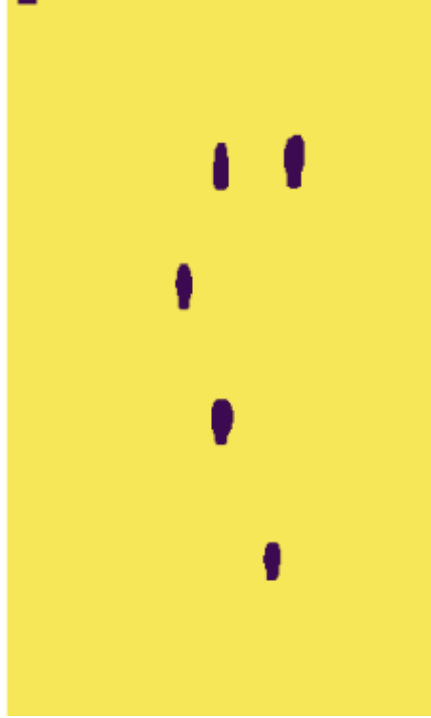
Illusory contours; a curious phenomenon where you see an object that appears to be occluding.

Background Subtraction

- ▶ If we know what the background looks like, it is easy to identify “interesting bits”
- ▶ Applications
 - ▶ Person in an office
 - ▶ Tracking cars on a road
 - ▶ Surveillance
- ▶ Approach:
 - ▶ use a **moving average** to estimate background image
 - ▶ subtract from current frame
 - ▶ large absolute values are interesting pixels
 - ▶ trick: use morphological operations to clean up pixels



Background Subtraction



Object tracking

Practice code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

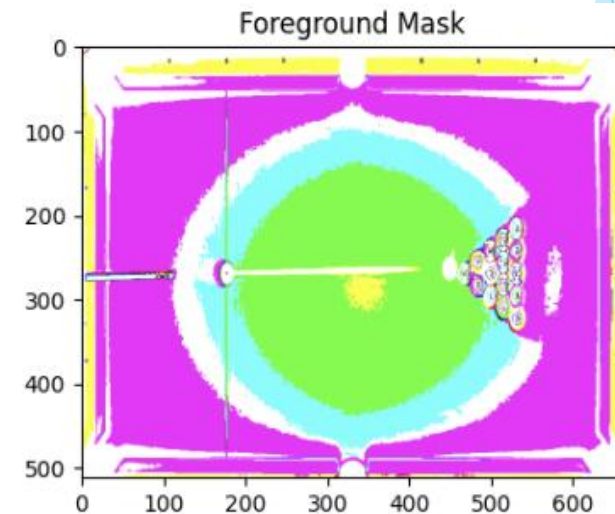
image = cv2.imread('billiard.jpg')
# Convert the image to grayscale
gray_image = image #cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Assume the background is a blank image or create a mask for
background
# Create a static background for subtraction
# (Here, we'll use a blank gray background of the same size)
background = np.full_like(gray_image, 128) # A gray background

# Perform background subtraction by taking the absolute difference
foreground = cv2.absdiff(gray_image, background)

# Threshold the foreground to get binary image (foreground mask)
_, foreground_mask = cv2.threshold(foreground, 30, 255,
cv2.THRESH_BINARY)
# Display the original image, background, and foreground mask
plt.figure(figsize=(15,5))
plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.subplot(1, 3, 2)
plt.title('Background ')
plt.imshow(background, cmap='gray')

plt.subplot(1, 3, 3)
plt.title('Foreground Mask')
plt.imshow(foreground_mask, cmap='gray')
plt.show()
```



Shot Boundary Detection

- Find the shots in a sequence of video
shot boundaries usually result in big differences between succeeding frames
- Strategy:
For each frame in an image sequence
Compute a distance between this frame and the previous frame
If the distance is larger then some threshold , classify the frame as a shot boundary

Shot : long sequences of video are composed of shots –much shorter subsequences within the same object.
Shot boundary detection : an important practical application of simple segmentation algorithm

Shot Boundary Detection

- ▶ Possible distances
 - ▶ frame differences
 - ▶ histogram differences
 - ▶ block comparisons
 - ▶ edge differences
- ▶ Applications:
 - ▶ representation for movies, or video sequences
 - ▶ find shot boundaries
 - ▶ obtain “most representative” frame

Background Subtraction

- ▶ The problem: Segment moving foreground objects from static background



from C. Stauffer and W. Grimson

Current image



Background image



Foreground pixels



courtesy of C. Wren

Pffinder



Practice code

```
import cv2
from google.colab.patches import cv2_imshow # Import cv2_imshow

# Create Background Subtractor object
background_subtractor = cv2.createBackgroundSubtractorMOG2()

# Open video capture (use 0 for webcam)
cap = cv2.VideoCapture('mp4_eg.mp4')

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Apply background subtraction
    foreground_mask = background_subtractor.apply(frame)

    cv2_imshow(frame) # Use cv2_imshow instead of cv2.imshow
    cv2_imshow(foreground_mask) # Use cv2_imshow instead of cv2.imshow

    # Press 'q' to exit the loop
    if cv2.waitKey(30) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



Algorithm

video sequence $I(\mathbf{x}, t)$ background $I_0(\mathbf{x}, t)$
frame difference $d(\mathbf{x}, t)$ thresholded frame diff $d_T(\mathbf{x}, t)$

for $t = 1:N$

 Update background model

 Compute frame difference

 Threshold frame difference

 Noise removal

end

Objects are detected where $d_T(\mathbf{x}, t)$ is non-zero

$$I_0(\mathbf{x}, t)$$

$$d(\mathbf{x}, t) = |I(\mathbf{x}, t) - I_0(\mathbf{x}, t)|$$

$$d_T(\mathbf{x}, t) = d(\mathbf{x}, t) > thresh$$

$$d_T(\mathbf{x}, t) = imerode(d_T(\mathbf{x}, t))$$

Background Modeling

- ▶ **Offline average**

- ▶ Pixel-wise mean values are computed during training phase (also called Mean and Threshold)

$$I_0(\mathbf{x}, t) = \frac{1}{T} \sum_{t=1}^T I(\mathbf{x}, t)$$

- ▶ **Adjacent Frame Difference**

- ▶ Each image is subtracted from previous image in sequence

$$I_0(\mathbf{x}, t) = I(\mathbf{x}, t - 1)$$

- ▶ **Moving average**

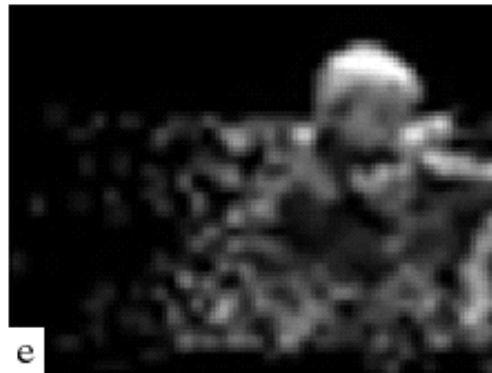
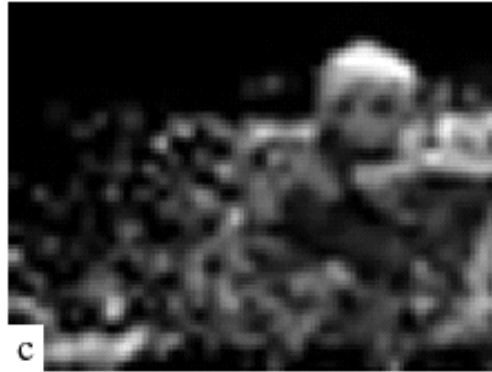
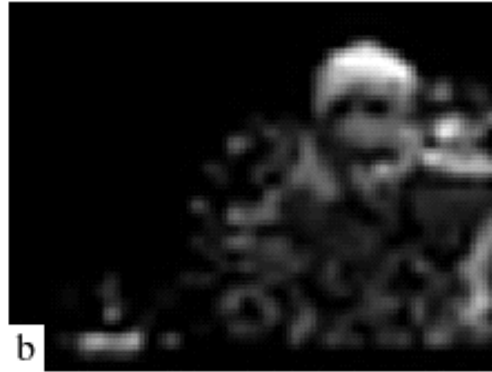
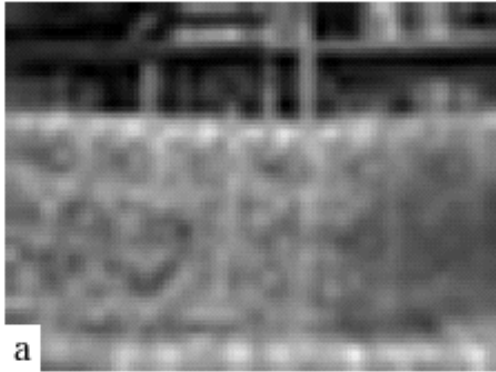
- ▶ Background model is linear weighted sum of previous frames

$$I_0(\mathbf{x}, t) = \frac{w_a I(\mathbf{x}, t) + \sum_{i=1}^N w_i I(\mathbf{x}, t - i)}{w_c}$$

Background Modeling

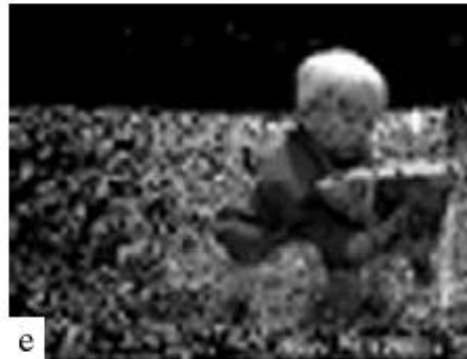
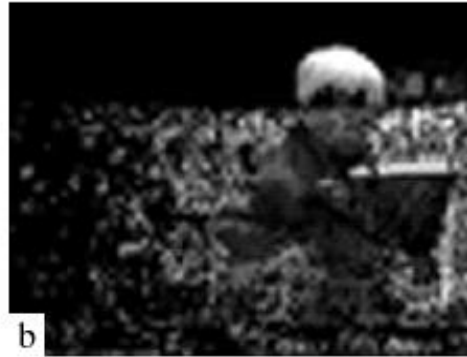


Background Modeling



80 x 60 frames

Background subtraction



a) Average for 120 frames

b) Difference pixels : average exceeds a threshold
-When the average crosses the threshold

c) Difference pixels : average exceeds an somewhat smaller threshold

d) Calculate the background using the Sophisticated method

e) Calculate the difference value from the background.

See caption, figure 14.11

An important point here is that background subtraction works quite badly in the presence of high spatial frequencies, because when we Estimate the background, we're almost always going to smooth it. Using 160 X 120 frames

160 x 120 frames

Background Subtraction: Issues

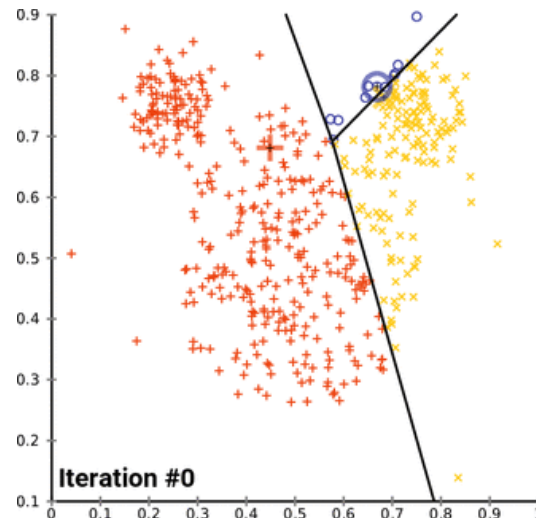
- ▶ **Dynamic Backgrounds**
- ▶ **Unimodal:** Pixel values vary over time even for static scenes
- ▶ **Multimodal:** Features in background can “oscillate”, requiring models which can represent disjoint sets of pixel values (e.g. waving trees against sky)
- ▶ **Shadows :** Moving objects can cast shadows on the background, which might be mistaken for foreground objects.
- ▶ **Lighting Variations/Continuous:** Gradual illumination changes alter the appearance of the background (e.g. time of day)
- ▶ **Discontinuous:** Sudden changes in illumination and other scene parameters alter the appearance of the background (e.g. flipping a light switch)
- ▶ **Camera Motion :** If the camera is moving or shaking, the background model may shift, leading to difficulties in differentiating between the background and foreground.

Segmentation using Simple Clustering Method

- ▶ Cluster together (pixels, tokens, etc.) that belong together
- ▶ **Agglomerative clustering**
 - ▶ Make each point separate cluster
 - ▶ Until the clustering is satisfactory
 - ▶ Merge the two clusters with the smallest inter-cluster distance
 - ▶ repeat
- ▶ **Divisive clustering**
 - ▶ Construct a single cluster containing all points
 - ▶ Until the clustering is satisfactory
 - ▶ Split the cluster that yields the two components with the largest inter-cluster distance
 - ▶ repeat

Clustering and Segmentation by K-means

At first a random set of cluster centres is initiated. Points are then assigned to their nearest centre. Centres are adjusted to match the centre of all points assigned to them. The assignment and adjustment steps are repeated until the centres no longer move.



Clustering and Segmentation by K-means

- ▶ Choose a fixed number of clusters
- ▶ Choose cluster centers and point-cluster allocations to minimize error
- ▶ can't do this by search, because there are too many possible allocations.
- ▶ Algorithm
 - ▶ fix cluster centers; allocate points to closest cluster
 - ▶ fix allocation; compute best cluster centers
- ▶ x could be any set of features for which we can compute a distance (careful about scaling)

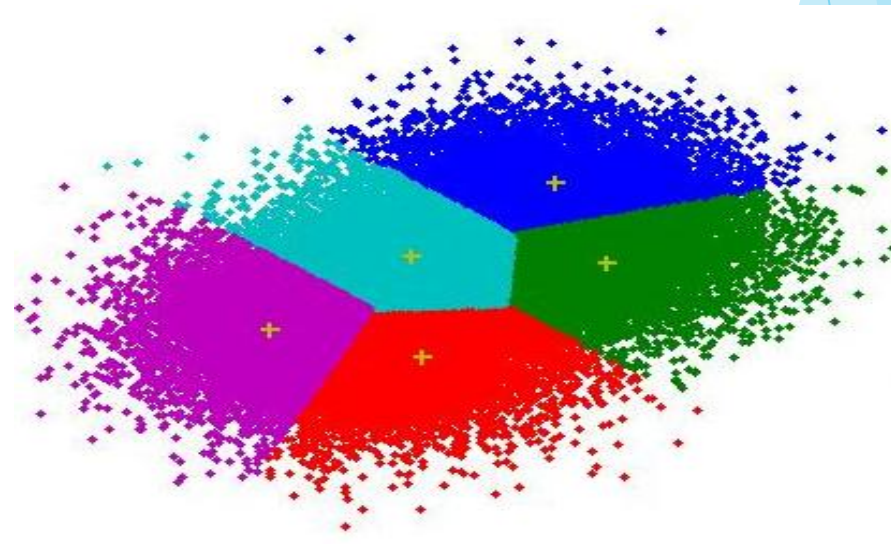
$$\sum_{i \in \text{clusters}} \left\{ \sum_{j \in \text{elements of } i\text{'th cluster}} \|x_j - \mu_i\|^2 \right\}$$

k-means clustering

- ▶ A method of cluster analysis which aims to partition n observations into k clusters in which each observation belongs to the cluster with the nearest mean.
- ▶ **Description:** Given a set of observations $(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$, where each observation is a d -dimensional real vector, k -means clustering aims to partition the n observations into k sets ($k \leq n$) $S = \{S_1, S_2, \dots, S_k\}$ so as to minimize the within-cluster sum of squares (WCSS):

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x}_j \in S_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\|^2$$

- ▶ where $\boldsymbol{\mu}_i$ is the mean of points in S_i .



K-Means Clustering

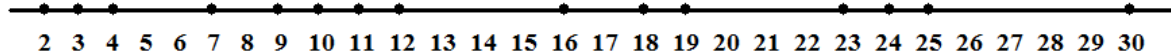
- ▶ Algorithms: Regarding computational complexity, the k-means clustering problem is:
 - ▶ NP-hard in general Euclidean space d even for 2 clusters
 - ▶ NP-hard for a general number of clusters k even in the plane
 - ▶ If k and d are fixed, the problem can be exactly solved in time $O(ndk+1 \log n)$, where n is the number of entities to be clustered
- ▶ Thus, a variety of heuristic algorithms are generally used.

Lloyd's algorithm (K-means Algorithm)

► The K-means algorithm steps:

- ① Choose the number of clusters, k .
- ② Randomly generate k clusters and determine the cluster centers, or directly generate k random points as cluster centers.
- ③ Assign each point to the nearest cluster center, where "nearest" is defined with respect to one of the distance measures discussed above.
- ④ Recompute the new cluster centers.
- ⑤ Repeat the two previous steps until some convergence criterion is met

Lloyd's algorithm (K-means Algorithm)



- ▶ Example1: Run K-means clustering with 3 clusters (initial centroids: 3, 16, 25) for at least 2 iterations

- ▶ Centroids:

- ▶ 3 - 2 3 4 7 9 new centroid: 5

- ▶ 16 - 10 11 12 16 18 19 new centroid: 14.33

- ▶ 25 - 23 24 25 30 new centroid: 25.5

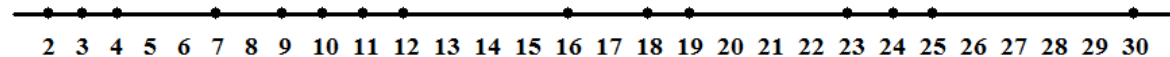
- ▶ Centroids:

- ▶ 5 - 2 3 4 7 9 new centroid: 5

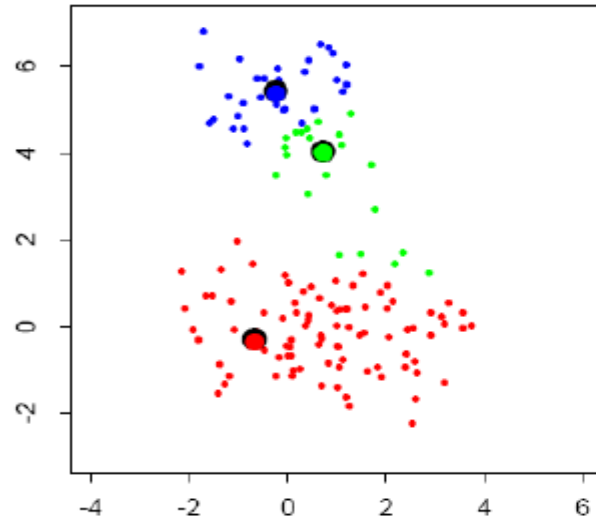
- ▶ 14.33 - 10 11 12 16 18 19 new centroid: 14.33

- ▶ 25.5 - 23 24 25 30 new centroid: 25.5

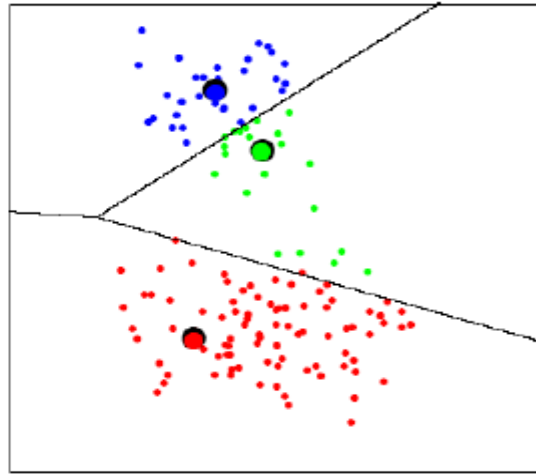
- ▶ Example2: Run K-means clustering with 3 clusters (initial centroids: 3, 12, 19) for at least 2 iterations



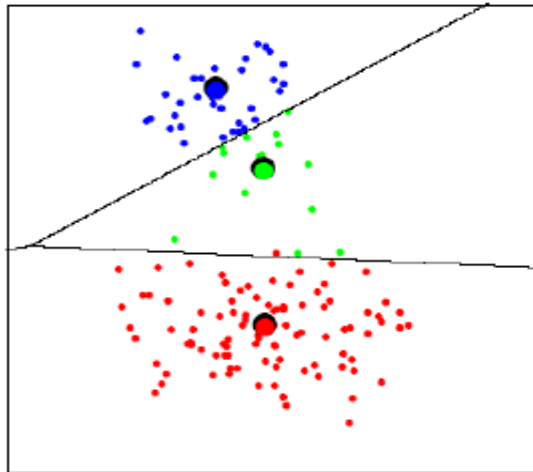
Initial Centroids



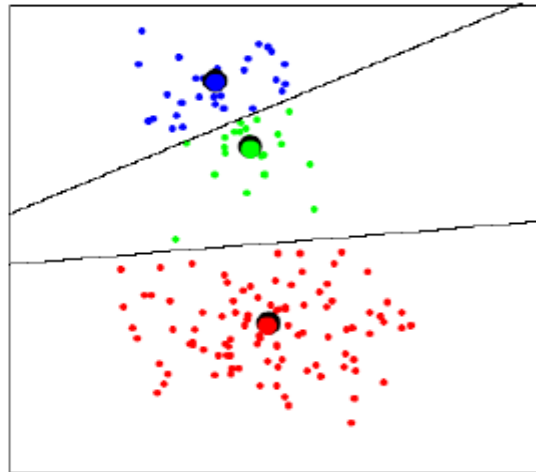
Initial Partition



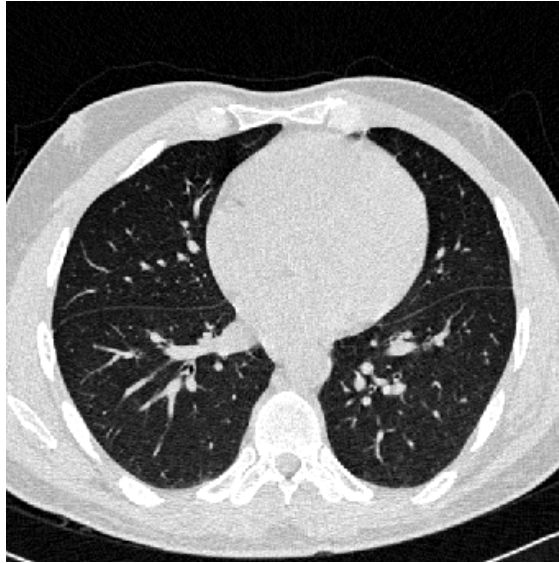
Iteration Number 2



Iteration Number 20



K-means Image Segmentation



An image (I)



Three-cluster image (J) on gray values of I

Matlab code:

```
I = double(imread('...'));
```

```
J = reshape(kmeans(I(:),3),size(I));
```

Note that K -means result is “noisy”

Clustering in Color Space

Each image point is mapped to a point in a color space, e.g.:

$$\text{Color}(i, j) = (R(i, j), G(i, j), B(i, j))$$

The points in the color space are grouped to clusters.
The clusters are then mapped back to regions in the image.

K-means Clustering

Image



Clusters on intensity



Clusters on color



K-means clustering using intensity alone and color alone

Here each pixel the mean intensity or mean color of its cluster --- this is basically just vector quantizing the image intensities/colors. Notice that there is no requirement that clusters be spatially localized and they're not.

K-means Clustering



Image



Clusters on color

K-means using color alone, 11 segments



K-means using
color alone,
11 segments.

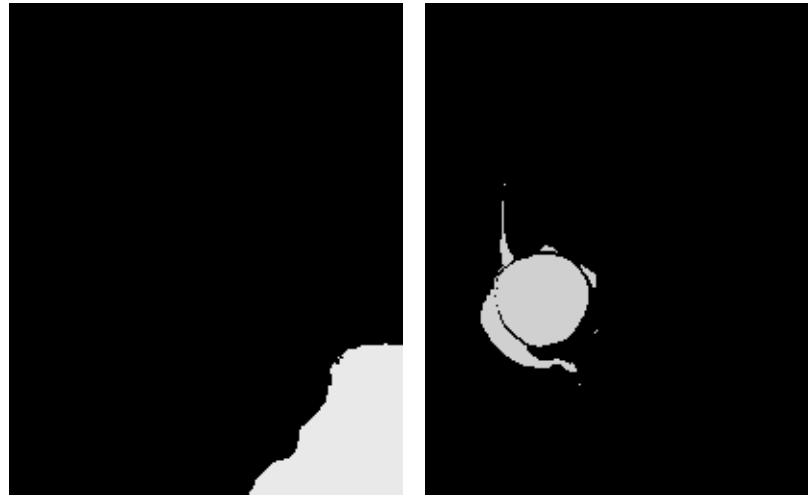


K-means Clustering



Here represented each pixel as (r, g, b, x, y) , which means that segments prefer to be spatially coherent. These are just some of 20 segments.

K-means using colour and position, 20 segments

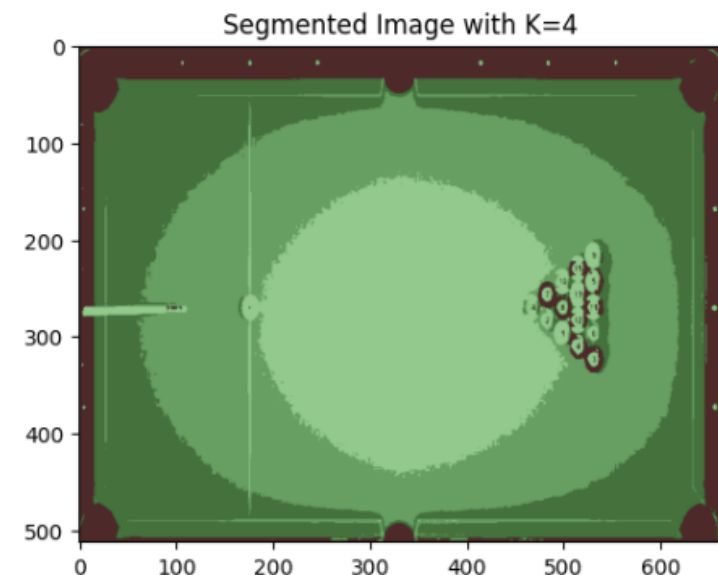


Practice code

```
import cv2
import numpy as np
from matplotlib import pyplot as plt

# Load image
image = cv2.imread('billiard.jpg')
# Convert the image from BGR (OpenCV default) to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
# Reshape the image into a 2D array of pixels and 3 color values (RGB)
pixel_values = image_rgb.reshape((-1, 3))
pixel_values = np.float32(pixel_values)
# Define stopping criteria and number of clusters (K)
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
K = 4 # Number of clusters
_, labels, centers = cv2.kmeans(pixel_values, K, None, criteria, 10,
cv2.KMEANS_RANDOM_CENTERS)

# Convert centers to 8-bit values (RGB)
centers = np.uint8(centers)
# Flatten the labels array
labels = labels.flatten()
# Convert all pixels to the color of their corresponding cluster center
segmented_image = centers[labels]
# Reshape back to the original image dimension
segmented_image = segmented_image.reshape(image_rgb.shape)
# Display the original and segmented image
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.imshow(image_rgb)
plt.title('Original Image')
plt.subplot(1, 2, 2)
plt.imshow(segmented_image)
plt.title(f'Segmented Image with K={K}')
plt.show()
```



Practice code

Load Image: The image is loaded using `cv2.imread()` and converted to RGB because OpenCV loads images in BGR format by default.

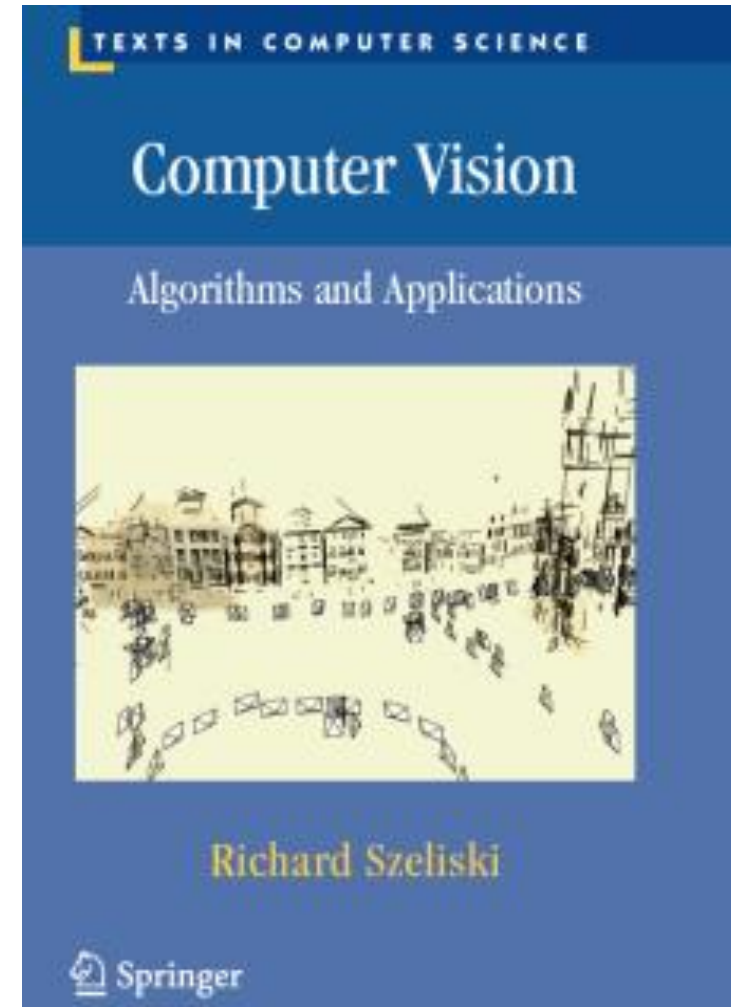
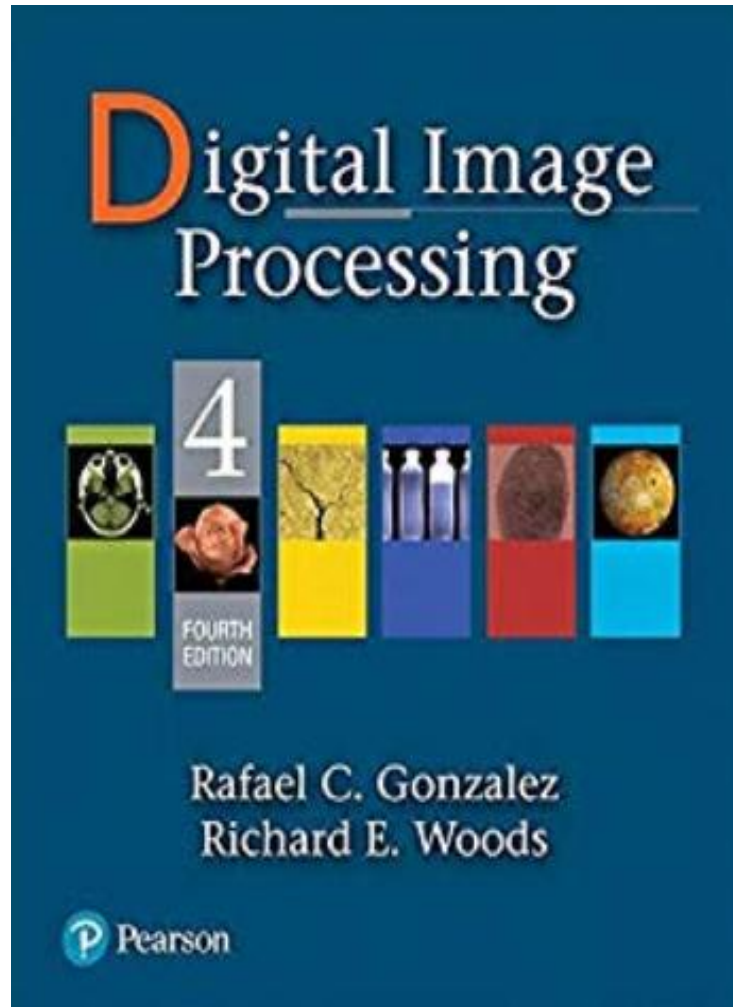
Reshape: The image is reshaped into a 2D array where each row is a pixel and each column is a color channel (R, G, B).

K-means: We apply K-means clustering to group the pixels into K clusters.

Segmentation: Each pixel is replaced by the centroid color of its cluster, giving the segmented image.

Visualization: Both the original and segmented images are displayed using Matplotlib.

Reference



Chapter -10 (gonzalez) and chapter-5 (Richard Szeliski book)

Question



Thank you

